If `policy` is `std::launch::async`, runs `INVOKE(fff,xyz...)` on its own thread. The returned `std::future` will become *ready* when this thread is complete and will hold either the return value or exception thrown by the function invocation. The destructor of the last future object associated with the asynchronous state of the returned `std::future` blocks until the future is *ready*.

If `policy` is `std::launch::deferred, fff` and `xyz...` are stored in the returned `std::future` as a deferred function call. The first call to the `wait()` or `get()` member functions on a future that shares the same associated state will execute `INVOKE(fff,xyz...)` synchronously on the thread that called `wait()` or `get()`.

The value returned or exception thrown by the execution of `INVOKE(fff,xyz...)` will be returned from a call to `get()` on that `std::future`.

If `policy` is `std::launch::async | std::launch::deferred` or the `policy` argument is omitted, the behavior is as-if either `std::launch::async` or `std::launch::deferred` had been specified. The implementation will choose the behavior on a call-by-call basis in order to take advantage of the available hardware concurrency without excessive oversubscription.

In all cases, the `std::async` call returns immediately.

Synchronization

The completion of the function invocation happens-before a successful return from a call to `wait(), get(), wait_for()`, or `wait_until()` on any `std::future` or `std::shared_future` instance that references the same associated state as the `std::future` object returned from the `std::async` call. In the case of a `policy` of `std::launch::async`, the completion of the thread on which the function invocation occurs also happens-before the successful return from these calls.